

General

General System Rule - The Main System is the official record of all player progress. The subsystems are only responsible for running mini games and sending the result to the Main System.

This means that the Main System must store:

- Player names
- Starter Pokemon
- Caught Pokemon
- Pokemon levels
- Coins
- Store purchases
- Inventory items
- Evolution records
- Game history

The subsystems must only send game results such as:

- Player name
- Game name
- Result
- Pokemon caught or affected
- Level gained
- Coins earned

Player Entry Flow

Open App



Enter Player Name



System checks if the name already exists



If name exists, load existing player progress



If name does not exist, create new player record



Open Player Dashboard

To avoid duplicate or confusing records, the system must follow these rules:

- [] Player name is required.
- [] Player name must be unique.
- [] Extra spaces must be removed before saving.
- [] Names should be checked case-insensitively.

Before continuing, show:

You are playing as: Jods

[Continue]

[Change Name]

Before saving any result, show:

Save this result to Jods?

Result: Squirtle caught

Reward: +20 coins

[Confirm Save]

[Cancel]

Recommended Architecture

Main System



Supabase Database

Subsystem 1: Catching Games



Supabase Database

Subsystem 2: Leveling Games



Supabase Database

Meaning, lahat sila connected sa **isang Supabase project**.

The Main System is still the official tracker, pero technically, all three apps can read/write to the same Supabase database.

Integration Guideline

Integration Rule

All subsystems must send results to the Main System using the same result format.

Standard format:

```
{  
  "playerName": "",  
  "pokemonName": "",  
  "gameName": "",  
  "result": "",  
  "levelGain": 0,  
  "coinsEarned": 0,  
  "sourceSystem": ""  
}
```

Standard Result Values

To avoid confusion, all developers must use the same result values.

For catching games:

caught

fled

For leveling games:

correct

wrong

partial

win

lose

For source systems:

main_system

catching_subsystem

leveling_subsystem

Manual_facilitator

20. Main System Response After Receiving Result

When the Main System receives a result, it must do the following:

1. Find the player using playerName.
2. If the player does not exist, create a new player record or return an error.
3. Find the Pokémon using pokemonName.
4. If result is caught, add Pokémon to player's Pokédex.
5. If levelGain is greater than 0, add levels to the selected Pokémon.
6. If coinsEarned is greater than 0, add coins to player wallet.
7. Save the action in game history.
8. Return success message to subsystem.

Example success message:

```
{  
  
  "success": true,  
  
  "message": "Result saved successfully."
```

```
}
```

Example error message:

```
{
```

```
  "success": false,
```

```
  "message": "Player not found."
```

```
}
```

Suggested Database Tables

players Table

players

- id
 - player_name
 - section
 - assigned_journey
 - starter_pokemon_id
 - created_at
-

pokemon Table

pokemon

- id
 - pokemon_name
 - type
 - region
 - image
 - evolution_stage
 - evolves_to
 - required_stone
-

player_pokemon Table

player_pokemon

- id
 - player_id
 - pokemon_id
 - level
 - source
 - status
 - created_at
-

wallets Table

wallets

- id
 - player_id
 - coin_balance
-

store_items Table

- store_items
- id
 - item_name
 - price
 - effect
 - compatible_type
-

inventory Table

- inventory
- id
 - player_id
 - item_id
 - quantity
-

game_logs Table

- game_logs
- id
 - player_id
 - pokemon_id
 - game_name
 - result
 - level_gain
 - coins_earned
 - source_system
 - logged_by
 - created_at

Final Complete System Flow

The final system should work like this:

1. Player opens the Pokédex App.
2. Player enters their name.
3. Main System creates or loads the player account.
4. Player answers starter personality quiz.
5. Starter Pokémon is saved to the player's Pokédex.
6. Player joins the Pokémon Journey.
7. Player plays catching mini games in Subsystem 1.
8. Subsystem 1 sends caught/fled result to Main System.
9. Player plays leveling mini games in Subsystem 2.
10. Subsystem 2 sends level/coin result to Main System.
11. Facilitator manually logs IRL game results in Main System.
12. Main System updates Pokémon, levels, coins, inventory, and history.
13. Player uses coins to buy items from Pokémon Store.
14. Player evolves Pokémon if requirements are met.
15. Final player progress is shown in the Pokédex.

Main System

Main System Development Guideline

The Main System is the Pokedex Mobile App. It is the main application where all player progress is stored and displayed.

The Main System must include:

- Player name entry
- Player dashboard
- Starter Pokemon selection
- Pokedex / Pokémon log
- Pokemon level tracking
- Coin wallet
- Pokemon Store
- Inventory
- Evolution feature
- Facilitator manual logging
- Game history
- Result receiver from subsystems

Main System Assignment

Developer

Assigned Features

Developer 1 Player name entry, player dashboard, player records

Developer 2 Starter selection and Pokédex

Developer 3 Coins, Pokémon Store, inventory, and evolution

Developer 4 Facilitator logging, game history, and subsystem integration

Main System Step-by-Step Development

Step 1: Create the Start Page

The Start Page is the first screen that appears when the app is opened.

Required elements:

- [] App title
- [] Player name input field
- [] Start Journey button

Example layout:

Welcome to PokéJourney!

Enter Player Name:

[Start Journey]

Expected behavior:

1. Player enters name.
2. Player clicks Start Journey.
3. System checks if the name already exists.
4. If the name exists, load existing progress.
5. If the name does not exist, create new player record.
6. Redirect player to dashboard.

Step 2: Create Player Record

When a new player enters their name, the system must create a player record.

Player data should include:

player_id
player_name
section
assigned_journey
starter_pokemon_id
created_at

Minimum required fields:

player_id
player_name
created_at

Optional fields:

section
assigned_journey
starter_pokemon_id

Step 3: Create Player Dashboard

After entering a name, the player should be redirected to their dashboard.

The dashboard should display:

Player Name
Starter Pokémon
Total Pokémon Caught
Total Coins
Total Level Points
Buttons to other pages

Example:

Player: Jods
Starter: Charmander
Coins: 75
Pokémon Caught: 4
Total Levels: 35

[Pokédex]
[Starter Selection]
[Pokémon Store]
[Inventory]
[Game History]

Step 4: Create Starter Selection

The player must be able to choose or receive a starter Pokémon.

Starter Pokémon choices:

Region	Grass	Fire	Water
Kanto	Bulbasaur	Charmander	Squirtle
Unova	Snivy	Tepig	Oshawott
Paldea	Sprigatito	Fuecoco	Quaxly

Starter selection flow:

Player answers questionnaire

↓

System calculates result

↓

System recommends starter Pokémon

↓

Player confirms starter

↓

Starter is saved under player account

↓

Starter is added to player's Pokédex

Important rule:

A player should only receive one starter Pokémon.

If the player already has a starter, show:

You already have a starter Pokémon: Charmander

Step 5: Create Pokémon Database

The system must have a list of available Pokémon.

Pokémon data should include:

pokemon_id
pokemon_name
type
region
image
evolution_stage
evolves_to
required_stone

Example:

pokemon_id: 001
pokemon_name: Bulbasaur
type: Grass
region: Kanto
evolution_stage: 1
evolves_to: Ivysaur
required_stone: Leaf Stone

Step 6: Create Player Pokédex

The Pokédex shows all Pokémon owned by the player.

Each Pokémon record should display:

Pokémon Name

Type

Region

Level

Source

Status

Example:

Charmander

Type: Fire

Region: Kanto

Level: 15

Source: Starter

Status: Active

Possible Pokémon sources:

Starter

PokeReflex

PokeGuess

IRL Catch

Manual Log

Step 7: Create Level Tracking

Each player-owned Pokémon must have its own level.

Level rules:

Result	Level Gain
Correct / Win	+10 levels
Wrong / Partial	+1 level
Lose	+0 level

Important rules:

- [] Players cannot manually edit levels.
- [] Levels can only be updated through game results or facilitator logging.
- [] Every level update must be recorded in game history.

Step 8: Create Coin Wallet

Each player must have a coin balance.

Suggested coin rewards:

Action	Coins
Successfully catch Pokémon	+20 coins
Win mini game	+15 coins
Partial attempt	+5 coins
Lose or failed attempt	+0 coins

Coin rules:

- [] Coins increase when player wins or completes a challenge.
- [] Coins decrease when player buys from the Pokémon Store.
- [] Coin balance cannot go below zero.

Step 9: Create Pokémon Store

The Pokémon Store allows players to buy items using coins.

Suggested store items:

Item	Price	Purpose
Fire Stone	50 coins	Used for Fire-type evolution
Water Stone	50 coins	Used for Water-type evolution
Leaf Stone	50 coins	Used for Grass-type evolution
Thunder Stone	60 coins	Used for Electric-type evolution
Rare Candy	40 coins	Adds bonus level

Store purchase flow:

Player selects item

↓

System checks if player has enough coins

↓

If enough, deduct coins

↓

Add item to inventory

↓

Record purchase in game history

If coins are not enough, show:

Not enough coins.

Step 10: Create Inventory

The inventory shows all items owned by the player.

Example:

Fire Stone x1
Water Stone x2

Inventory rules:

- [] Purchased items must appear in inventory.
- [] Item quantity should increase when bought.
- [] Item quantity should decrease when used.

Step 11: Create Evolution Feature

The system should allow Pokémon to evolve.

Simple evolution rule:

A Pokémon can evolve if:

1. It reaches the required level.
2. The player owns the required evolution stone.

Suggested evolution requirements:

Evolution Stage	Requirement
First evolution	Level 20 + correct stone
Final evolution	Level 40 + correct stone

Type-based stone rule:

Pokémon Type	Required Stone
Fire	Fire Stone
Water	Water Stone
Grass	Leaf Stone

Electric

Thunder Stone

Evolution flow:

Player selects Pokémon



System checks level requirement



System checks required stone



If valid, Pokémon evolves



Stone quantity decreases



Evolution is recorded in game history

If not valid, show:

This Pokémon cannot evolve yet.

Step 12: Create Facilitator Manual Logging

The Facilitator Manual Logging feature is used for IRL games.

Facilitator should be able to log:

Player name
Game name
Result
Pokémon involved
Level gain
Coins earned
Notes

Manual logging examples:

Player: Jods
Game: IRL Catch a Pokémon
Result: Caught
Pokémon: Pikachu
Reward: +20 coins
Player: Jods
Game: Pokémon Showdown Battle
Result: Win
Pokémon: Charmander
Reward: +10 levels, +20 coins

Manual logging must update:

[] Player's Pokédex, if Pokémon was caught
[] Pokémon level, if level was earned
[] Coin wallet, if coins were earned
[] Game history

Step 13: Create Game History

Game history records every important action.

Examples of history logs:

Jods selected Charmander as starter.

Jods caught Squirtle through PokeReflex.

Jods gained +10 levels from Battle Predictor.

Jods earned +15 coins.

Jods bought Fire Stone.

Jods evolved Charmander.

Game history should include:

history_id
player_id
pokemon_id
game_name
result
level_gain
coins_earned
source_system
created_at

Step 14: Create Result Receiver for Subsystems

The Main System must receive results from Subsystem 1 and Subsystem 2.

Recommended result format:

```
{
  "playerName": "Jods",
  "pokemonName": "Squirtle",
  "gameName": "PokeReflex",
  "result": "caught",
  "levelGain": 0,
  "coinsEarned": 20,
  "sourceSystem": "catching_subsystem"
}
```

For leveling games:

```
{
  "playerName": "Jods",
  "pokemonName": "Charmander",
  "gameName": "Battle Predictor",
  "result": "correct",
  "levelGain": 10,
  "coinsEarned": 15,
  "sourceSystem": "leveling_subsystem"
}
```

The Main System must:

- [] Find the player by playerName.
- [] Update Pokémon record if needed.
- [] Update level if needed.
- [] Update coins if needed.
- [] Save the result in game history.

Subsystem 1 (catch)

Subsystem 1 Development Guideline

Catching Mini Games

Subsystem 1 Description

Subsystem 1 is responsible for catching Pokémon through mini games.

It includes:

1. **PokeReflex**
2. **PokeGuess**
3. **Wild Pokémon generation**
4. **Sending catch result to Main System**

Subsystem 1 Assignment

Developer	Assigned Features
Developer 1	Player name entry and wild Pokémon generation
Developer 2	PokeReflex mini game
Developer 3	PokeGuess mini game and Main System integration

Subsystem 1 Step-by-Step Development

Step 1: Create Player Name Entry

Subsystem 1 should also ask for the player name.

Flow:

Open Subsystem 1

↓

Enter Player Name

↓

Confirm Player Name

↓

Select Catching Mini Game

Example:

Enter Player Name:

Jods

You are playing as Jods.

[Continue]

Important:

The player name must match the player name used in the Main System.

Step 2: Create Mini Game Selection Page

The player should choose which catching mini game to play.

Options:

☐ PokeReflex

☐ PokeGuess

Flow:

Enter Player Name



Select Mini Game



Generate Wild Pokémon



Start Game

Step 3: Generate Wild Pokémon

Before the game starts, the system should generate a wild Pokémon.

Example:

A wild Squirtle appeared!

Wild Pokémon data should include:

pokemon_name

type

region

image

difficulty

Suggested difficulty levels:

Difficulty	Description
Easy	Common or starter Pokémon
Medium	Less common Pokémon
Hard	Rare Pokémon

Step 4: Build PokeReflex

PokeReflex is a reaction-based catching game.

Game concept:

A target word appears on the screen.

The player must click the matching symbol.

Example (but not limited to):

Target: Fire Type

Choices:

[Water Symbol]

[Fire Symbol]

[Leaf Stone]

[Poké Ball]

Correct answer:

Fire Symbol

PokeReflex flow:

Wild Pokémon appears

↓

Target word appears

↓

Choices appear

↓

Player clicks answer

↓

System checks if answer is correct

↓

If correct, Pokémon is caught

↓

If wrong, Pokémon fled

↓

Result is sent to Main System

PokeReflex requirements:

[] Show wild Pokémon.

[] Show target word.

[] Show answer choices.

[] Check selected answer.

[] Display result.

[] Send result to Main System.

PokeReflex reward rules:

Result

Reward

Correct

**Pokémon caught +
coins**

Wrong

**Pokémon fled + no
coins**

Recommended coins:

Result	Coins
---------------	--------------

Correct	+20 coins
----------------	------------------

Wrong	+0 coins
--------------	-----------------

Step 5: Build PokeGuess

PokeGuess is a guessing-based catching game.

Game concept:

The system gives a category or clue.

The player guesses the correct Pokémon.

Example:

Category: Fire-type starter

Clue: I am from Paldea and I look like a crocodile.

Choices:

[Charmander]

[Tepig]

[Fuecoco]

[Squirtle]

Correct answer:

Fuecoco

PokeGuess flow:

Wild Pokémon appears

↓

System shows category or clue

↓

Player chooses an answer

↓

System checks answer

↓

If correct, Pokémon is caught

↓

If wrong, Pokémon fled

↓

Result is sent to Main System

PokeGuess requirements:

[] Generate category or clue.

[] Show answer choices.

[] Check answer.

[] Display result.

[] Send result to Main System.

PokeGuess reward rules:

Result	Reward
Correct	Pokémon caught + coins
Wrong	Pokémon fled + no coins

Recommended coins:

Result	Coins
Correct	+20 coins
Wrong	+0 coins

Step 6: Send Catch Result to Main System

After each catching mini game, Subsystem 1 must send the result to the Main System.

Caught result example:

```
{  
  "playerName": "Jods",  
  "pokemonName": "Squirtle",  
  "gameName": "PokeReflex",  
  "result": "caught",  
  "levelGain": 0,  
  "coinsEarned": 20,  
  "sourceSystem": "catching_subsystem"  
}
```

Fled result example:

```
{  
  "playerName": "Jods",  
  "pokemonName": "Squirtle",  
  "gameName": "PokeReflex",  
  "result": "fled",  
  "levelGain": 0,  
  "coinsEarned": 0,  
  "sourceSystem": "catching_subsystem"  
}
```

Subsystem 1 Final Checklist

- ☐ Player name entry
- ☐ Player name confirmation
- ☐ Mini game selection
- ☐ Wild Pokémon generation
- ☐ PokeReflex mini game
- ☐ PokeGuess mini game
- ☐ Result screen
- ☐ Send caught/fled result to Main System
- ☐ Main System updates player Pokédex
- ☐ Main System updates coins
- ☐ Main System records game history

Subsystem 2 (levels)

Subsystem 2 Development Guideline

Leveling Mini Games

Subsystem 2 Description

Subsystem 2 is responsible for leveling up the player's Pokémon through mini games.

It includes:

1. **Battle Predictor**
2. **Guess That Pokémon**
3. **Match That Pokémon**
4. **Sending level and coin rewards to Main System**

The Pokémon Showdown battle is an IRL activity, so it will be logged manually in the Main System by the facilitator.

Subsystem 2 Team Assignment

There are three developers assigned to Subsystem 2.

Recommended task distribution:

Developer	Assigned Features
Developer 1	Player name entry, Pokémon selection, Battle Predictor
(Carlos) Developer 2	Guess That Pokémon

Developer 3 Match That Pokémon and Main System integration

Subsystem 2 Step-by-Step Development

Step 1: Create Player Name Entry

Subsystem 2 should ask for the player name.

Flow:

Open Subsystem 2

↓

Enter Player Name

↓

Confirm Player Name

↓

Select Pokémon to Level Up

↓

Select Leveling Mini Game

Example:

Enter Player Name:

Jods

You are playing as Jods.

[Continue]

Step 2: Select Pokémon to Level Up

Before playing a leveling mini game, the player must select which Pokémon will receive the level reward.

Flow:

Enter Player Name

↓

System loads player's Pokémon

↓

Player selects Pokémon

↓

Player selects mini game

↓

Start game

Example:

Player: Jods

Choose Pokémon to level up:

[Charmander - Level 15]

[Squirtle - Level 8]

[Pikachu - Level 12]

Important:

The level reward must only be applied to the selected Pokémon.

Step 3: Create Mini Game Selection Page

The player should choose which leveling mini game to play.

Options:

- ☐ Battle Predictor
- ☐ Guess That Pokémon
- ☐ Match That Pokémon

Step 4: Build Battle Predictor

Battle Predictor is a type advantage prediction game.

Game concept:

The system shows two Pokémon.

The first Pokémon is the opponent.

The second Pokémon is the challenger.

The player must predict if the challenger will win based on type advantage.

Example:

Opponent: Bulbasaur

Type: Grass

Challenger: Charmander

Type: Fire

Question:

Will Charmander likely win based on type advantage?

Choices:

[Yes]

[No]

Correct answer:

Yes

Battle Predictor flow:

System generates opponent Pokémon

↓

System generates challenger Pokémon

↓

System shows both Pokémon and their types

↓

Player predicts Yes or No



System checks type advantage



System displays correct or wrong result



Result is sent to Main System

Battle Predictor requirements:

- [] Generate opponent Pokémon.
- [] Generate challenger Pokémon.
- [] Show both Pokémon types.
- [] Ask player to predict winner.
- [] Check type advantage.
- [] Display result.
- [] Send level and coin reward to Main System.

Battle Predictor reward rules:

Result	Level Gain	Coins
Correct	+10 levels	+15 coins
Wrong	+1 level	+5 coins

Step 5: Build Guess That Pokémon

Guess That Pokémon is an image guessing game.

Game concept:

The system shows a blurred Pokémon image.

The player must guess:

1. Pokémon name
2. Pokémon type
3. Counter type

Example:

Blurred Image: Squirtle

Questions:

Pokémon Name: _____

Pokémon Type: _____

Counter Type: _____

Correct answers:

Name: Squirtle

Type: Water

Counter Type: Grass or Electric

Scoring:

Answer	Points
Correct Pokémon name	5 points
Correct Pokémon type	3 points

Correct counter type	2 points
Total	10 points

Reward rules:

Score	Level Gain	Coins
8–10 points	+10 levels	+15 coins
4–7 points	+5 levels	+8 coins
1–3 points	+1 level	+3 coins
0 points	+0 levels	+0 coins

Guess That Pokémon requirements:

- ☐ Show blurred Pokémon image.
- ☐ Ask for Pokémon name.
- ☐ Ask for Pokémon type.
- ☐ Ask for counter type.
- ☐ Compute score.
- ☐ Display result.
- ☐ Send level and coin reward to Main System.

Step 6: Build Match That Pokémon

Match That Pokémon is a memory card matching game.

Game concept:

Cards are shown face down.

The player flips two cards at a time.

If the cards match, they stay open.

If the cards do not match, they close again.

The player must match pairs to earn rewards.

Recommended matching pairs:

Charmander + Fire Symbol

Squirtle + Water Symbol

Bulbasaur + Grass Symbol

Pikachu + Thunder Stone

Match That Pokémon flow:

Cards appear face down



Player flips first card



Player flips second card



System checks if pair matches



Matched cards stay open



Unmatched cards close again



System counts matched pairs

↓
Reward is based on number of matched pairs
↓
Result is sent to Main System

Reward rules:

Result	Level Gain	Coins
2 matched pairs	+10 levels	+15 coins
1 matched pair	+3 levels	+5 coins
0 matched pairs	+0 levels	+0 coins

Match That Pokémon requirements:

- ☐ Display face-down cards.
- ☐ Allow player to flip cards.
- ☐ Check if cards match.
- ☐ Keep matched cards open.
- ☐ Close unmatched cards.
- ☐ Count matched pairs.
- ☐ Display result.
- ☐ Send level and coin reward to Main System.

Step 7: Send Level Result to Main System

After each leveling mini game, Subsystem 2 must send the result to the Main System.

Correct result example:

```
{
  "playerName": "Jods",
  "pokemonName": "Charmander",
  "gameName": "Battle Predictor",
  "result": "correct",
  "levelGain": 10,
  "coinsEarned": 15,
  "sourceSystem": "leveling_subsystem"
}
```

Wrong result example:

```
{
  "playerName": "Jods",
  "pokemonName": "Charmander",
  "gameName": "Battle Predictor",
  "result": "wrong",
  "levelGain": 1,
  "coinsEarned": 5,
  "sourceSystem": "leveling_subsystem"
}
```

Subsystem 2 Final Checklist

[] Player name entry

- [] Player name confirmation
- [] Load player's Pokémon
- [] Select Pokémon to level up
- [] Mini game selection
- [] Battle Predictor
- [] Guess That Pokémon
- [] Match That Pokémon
- [] Result screen
- [] Send level/coin result to Main System
- [] Main System updates Pokémon level
- [] Main System updates coins
- [] Main System records game history